

Injection de dépendances

Rezak AZIZ
rezak.aziz@lecnam.net

CNAM Paris

Contexte général

Une application logicielle est composée de plusieurs composants (classes, modules ou services) qui interagissent afin de fournir une fonctionnalité globale.

Mise en contexte : architecture logicielle

Contexte général

Une application logicielle est composée de plusieurs composants (classes, modules ou services) qui interagissent afin de fournir une fonctionnalité globale.

Observation

Ces composants dépendent souvent les uns des autres pour fonctionner.

Interface utilisateur



Services métier



Accès aux données

```
graph TD; A[Interface utilisateur] --> B[Services métier]; B --> C[Accès aux données];
```

Le problème des dépendances

Problème fondamental

Une mauvaise gestion des dépendances entraîne un couplage excessif entre les composants du système.

Le problème des dépendances

Problème fondamental

Une mauvaise gestion des dépendances entraîne un couplage excessif entre les composants du système.

Conséquences

- maintenance difficile ;
- faible évolutivité ;
- tests unitaires complexes ;
- réutilisabilité limitée.

Notion de couplage

Définition

Une classe **A** est dite couplée à une classe **B** lorsque le code de **A** dépend explicitement de l'implémentation de **B**.

Notion de couplage

Définition

Une classe **A** est dite couplée à une classe **B** lorsque le code de **A** dépend explicitement de l'implémentation de **B**.

Définition

Une classe **A** est dite **fortement** couplée à une classe **B** lorsque **A** est couplée avec **B** et **B** est couplée avec **A**.

Notion de couplage

Définition

Une classe **A** est dite couplée à une classe **B** lorsque le code de **A** dépend explicitement de l'implémentation de **B**.

Définition

Une classe **A** est dite **fortement** couplée à une classe **B** lorsque **A** est couplée avec **B** et **B** est couplée avec **A**.

Dépendances en chaîne

Classe A $\rightarrow$ Classe B $\rightarrow$ Classe C

A dépend indirectement de C

Notion de couplage

Définition

Une classe **A** est dite couplée à une classe **B** lorsque le code de **A** dépend explicitement de l'implémentation de **B**.

Définition

Une classe **A** est dite **fortement** couplée à une classe **B** lorsque **A** est couplée avec **B** et **B** est couplée avec **A**.

Dépendances en chaîne

Classe A $\rightarrow$ Classe B $\rightarrow$ Classe C

A dépend indirectement de C

Couplage

A est couplée à B $\Leftrightarrow$ pour compiler A il faut compiler B.

Toute modification de **B** impacte directement **A**.

Problématique

Comment concevoir un logiciel faiblement couplé tout en conservant une architecture claire et maintenable ?

Pourquoi réduire le couplage ?

Problématique

Comment concevoir un logiciel faiblement couplé tout en conservant une architecture claire et maintenable ?

Pourquoi réduire le couplage ?

- Diviser pour régner
- Simplifier la création de tests unitaires.
- Simplifier le travail en équipe.

Exemple de couplage

Code source

```
public class LogicielDeCalcul {  
    private FileLogger logger;  
  
    public LogicielDeCalcul() {  
        logger = new FileLogger("log.txt");  
    }  
}
```

Analyse

Exemple de couplage

Code source

```
public class LogicielDeCalcul {  
    private FileLogger logger;  
  
    public LogicielDeCalcul() {  
        logger = new FileLogger("log.txt");  
    }  
}
```

Analyse

- dépendance concrète ;
- instantiation interne ;
- choix figé ;
- testabilité réduite.

Section 1

Principe inversion de Dépendance

Principe d'inversion de dépendances

Objectif

Réduire le couplage entre les composants d'un système logiciel en supprimant les dépendances directes aux implémentations concrètes.

Principe d'inversion de dépendances

Objectif

Réduire le couplage entre les composants d'un système logiciel en supprimant les dépendances directes aux implémentations concrètes.

Idée centrale

Un composant ne doit pas dépendre d'un autre composant concret, mais d'une abstraction.

Composant métier



Abstraction



Implémentation

Dependency Inversion Principle (SOLID)

Énoncé du principe

- Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau.
- Les abstractions ne doivent pas dépendre des détails. Les détails doivent dépendre des abstractions.

Conséquence

Les abstractions définissent les contrats de collaboration entre les composants.

Dependency Inversion Principle (SOLID)

Énoncé du principe

- Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau.
- Les abstractions ne doivent pas dépendre des détails. Les détails doivent dépendre des abstractions.

Conséquence

Les abstractions définissent les contrats de collaboration entre les composants.

Composant métier



Abstraction



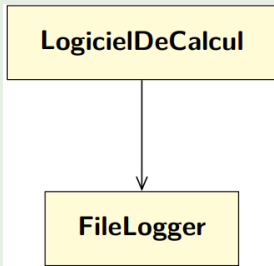
Implémentation

Programmer contre des interfaces

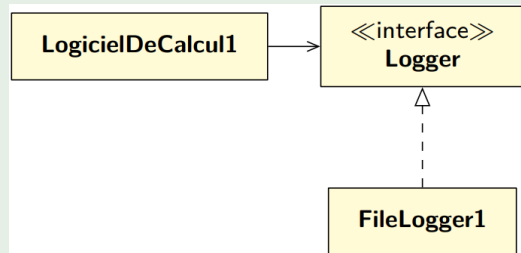
Principe

Une interface définit ce que fait un composant, sans préciser comment il le fait.

Avant DIP



Après DIP



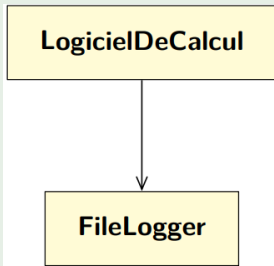
Avantages

Programmer contre des interfaces

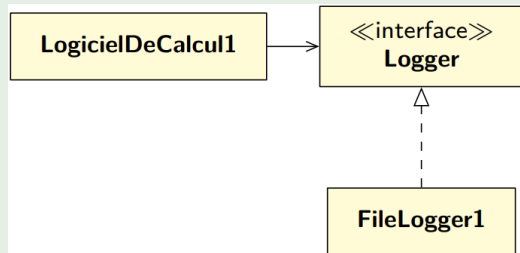
Principe

Une interface définit ce que fait un composant, sans préciser comment il le fait.

Avant DIP



Après DIP



Avantages

- découplage du code ; interchangeabilité des implémentations ; meilleure évolutivité.

Exemple : abstraction par interface

Interface

```
public interface Logger {  
    void log(String message);  
}
```

Implémentation

```
public class FileLogger  
    implements Logger {  
    ...  
}
```

Analyse

- le contrat est explicite ;
- plusieurs implémentations possibles ;
- le code client est indépendant.

Cas problématique

```
public class LogicielDeCalcul {  
  
    private Logger logger;  
  
    public LogicielDeCalcul() {  
        logger = new FileLogger("log.txt");  
    }  
}
```

Limite de l'interface seule

Problème

Utiliser une interface ne suffit pas si le code client instancie directement une implémentation concrète.

Conséquence

Le couplage est simplement déplacé, mais non supprimé.

Cas problématique

```
public class LogicielDeCalcul {  
  
    private Logger logger;  
  
    public LogicielDeCalcul() {  
        logger = new FileLogger("log.txt");  
    }  
}
```

Section 2

Principe inversion de Controle

Rappel : inversion de dépendances

- Réduction du couplage entre composants ;
- Dépendance vers des abstractions ;
- Séparation entre contrat et implémentation.

Limite identifiée

- Le DIP ne précise pas :
 - ▶ qui contrôle l'exécution ;
 - ▶ quand les composants sont appelés.
- Le problème du contrôle reste entier.

Question fondamentale

Qui doit être responsable du contrôle du déroulement global de l'application ?

Enjeu architectural

- Maintenabilité du système ;
- Évolutivité ;
- Séparation claire des responsabilités.

Fonctionnement

Dans une architecture traditionnelle, le composant métier :

- crée ses dépendances ;
- appelle directement les autres composants ;
- contrôle l'ordre d'exécution.

Conséquences

- Couplage fort ;
- Logique métier mêlée à l'orchestration ;
- Architecture difficile à faire évoluer.

Principe d'inversion du contrôle

Définition

L'inversion du contrôle est un principe architectural selon lequel le contrôle du flux d'exécution n'est plus assuré par le composant métier, mais délégué à une infrastructure externe.

Principe d'inversion du contrôle

Définition

L'inversion du contrôle est un principe architectural selon lequel le contrôle du flux d'exécution n'est plus assuré par le composant métier, mais délégué à une infrastructure externe.

Idée centrale

- Le composant métier devient passif ;
- Il ne décide plus quand il est exécuté ;
- Il se concentre sur sa logique fonctionnelle.

Avant IoC

- Orchestration locale ;
- Dépendances figées ;
- Flux d'exécution implicite.

Après IoC

- Orchestration centralisée ;
- Composants indépendants ;
- Flux d'exécution explicite.

Sans inversion du contrôle

Composant métier → contrôle du flux

Avec inversion du contrôle

Infrastructure → orchestration

Caractéristique

Le composant connaît le déroulement global de l'application.

Caractéristique

Le composant ignore le contexte global dans lequel il est exécuté.

Formulation

Don't call us, we'll call you

Interprétation

- Le composant n'appelle pas l'infrastructure ;
- Il expose des points d'extension ;
- Il est invoqué au moment opportun.

Observation

Les frameworks :

- contrôlent le cycle de vie ;
- orchestrent l'exécution ;
- appellent le code applicatif.

Conclusion

Un framework n'est pas une bibliothèque. Il impose un cadre d'exécution basé sur l'IoC.

Section 3

Injection de dépendance

Rappel

L'inversion du contrôle confie le contrôle du flux d'exécution à une infrastructure externe.

Question

Comment fournir concrètement les dépendances aux composants sans qu'ils les créent eux-mêmes ?

Injection de dépendances : définition

Définition

L'injection de dépendances est un mécanisme de conception dans lequel les dépendances d'un objet sont fournies depuis l'extérieur, plutôt que créées par l'objet lui-même.

Objectif

- Supprimer la responsabilité de création des dépendances ;
- Compléter l'inversion du contrôle.

Comparaison conceptuelle

Sans injection

Objet $\rightarrow$ new Dépendance()

Avec injection

Objet $\leftarrow$ Dépendance

Caractéristique

L'objet contrôle la création de ses dépendances.

Caractéristique

Les dépendances sont fournies par une entité externe.

Rôle de l'injection de dépendances

Responsabilités supprimées

- Création des objets ;
- Choix des implémentations ;
- Configuration technique.

Responsabilité conservée

- Logique métier ;
- Règles fonctionnelles ;
- Utilisation des dépendances.

Mécanismes principaux

- Injection par constructeur ;
- Injection par setter ;
- Approche mixte.

Critère de choix

Le mécanisme dépend du caractère obligatoire ou optionnel de la dépendance.

Injection par constructeur

Principe

Les dépendances sont fournies au moment de la création de l'objet.

Exemple

```
public class LogicielDeCalcul {  
    private Logger logger;  
  
    public LogicielDeCalcul(Logger logger) {  
        this.logger = logger;  
    }  
}
```

Caractéristiques

- Dépendances explicites ;
- Objet valide dès sa création ;
- Approche recommandée.

Injection par setter

Principe

Les dépendances sont fournies après la création de l'objet via des méthodes dédiées.

Exemple

```
public class LogicielDeCalcul {  
    private Logger logger;  
  
    public void setLogger(Logger logger) {  
        this.logger = logger;  
    }  
}
```

Attention

- Objet partiellement initialisé ;
- Dépendances potentiellement absentes.
- A privilégier uniquement si une dépendance par défaut est incluse dans le constructeur.

Principe

Une dépendance par défaut est fournie lors de la construction, mais peut être remplacée ultérieurement.

Intérêt

Compromis entre sûreté et flexibilité.

Exemple

```
public class LogicielDeCalcul {  
    private Logger logger;  
  
    public LogicielDeCalcul(Logger logger) {  
        this.logger = logger;  
    }  
  
    public void setLogger(Logger logger) {  
        this.logger = logger;  
    }  
}
```

Synthèse

- Constructeur :
 - ▶ sûr ;
 - ▶ explicite ;
 - ▶ recommandé.
- Setter :
 - ▶ flexible ;
 - ▶ moins sûr.

Règle pratique

Une dépendance indispensable doit être injectée par le constructeur.

Constat

Dans une application réelle, la création manuelle des objets devient rapidement complexe :

- nombreuses dépendances ;
- dépendances en cascade ;
- configurations variables.

Limite

Le code d'assemblage devient difficile à maintenir et à faire évoluer.

Assemblage manuel

```
new A(new B(new C()))  
new A(new B(new D()))
```

...

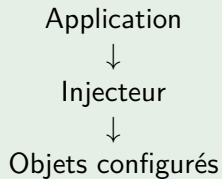
Rôle d'un framework d'injection

Responsabilités principales

Un framework d'injection de dépendances :

- crée les objets ;
- résout les dépendances ;
- injecte les composants appropriés ;
- gère le cycle de vie des objets.

Vue conceptuelle



Notion d'assembleur

Définition

L'assembleur est le composant chargé de construire et de relier les objets de l'application.

Évolution

Dans un framework, l'assembleur est automatisé et configurable.

Responsabilité

Création
Injection
Configuration

Principe

Le framework doit savoir quelle implémentation associer à chaque abstraction.

Approches courantes

- configuration externe (fichiers) ;
- annotations dans le code ;
- conventions.

Exemples de frameworks

Frameworks connus

- Spring (écosystème Java) ;
- Guice (léger, focalisé sur la DI).

Remarque

Ces frameworks vont bien au-delà de l'injection de dépendances.

Positionnement

DI = fondation

Framework = automatisation

Ce que fait réellement le framework

Important

Un framework d'injection :

- n'améliore pas un mauvais design ;
- n'est utile que si les principes sont respectés.

Attention

Utiliser un framework sans comprendre l'injection de dépendances conduit à du code complexe et fragile.

À retenir

- La DI est un mécanisme ;
- Elle implémente l'loC ;
- Elle améliore la testabilité ;
- Elle prépare l'usage des frameworks.

Message clé

Un bon design est un code qui ne sait pas comment ses dépendances sont créées.

Section 4

Injection de dépendances et testabilité

Problème

Un code fortement couplé est difficile à tester de manière isolée, car il dépend de ressources externes (fichiers, bases de données, réseau).

Conséquence

Les tests deviennent lents, fragiles et peu reproductibles.

Apport de l'injection

L'injection de dépendances permet de :

- remplacer une dépendance réelle par une dépendance simulée ;
- isoler le composant métier ;
- tester uniquement son comportement.

Dummy et Mock

Contexte

Dans le cadre de l'injection de dépendances, il est souvent nécessaire de remplacer des dépendances réelles par des objets simplifiés lors des tests.

Dummy

- Objet minimal ;
- Ne contient aucun comportement utile ;
- Sert uniquement à satisfaire une dépendance.

Usage typique

Tester un composant sans déclencher d'effet de bord (fichier, réseau, base de données).

Mock

- Objet simulé ;
- Comportement contrôlé ;
- Permet de vérifier des interactions.

Usage typique

Vérifier qu'un composant appelle correctement une dépendance ou respecte un contrat.

Exemple : Dummy simple

Implémentation

```
Logger dummyLogger = msg -> {};  
LogicielDeCalcul logiciel =  
    new LogicielDeCalcul(dummyLogger);
```

Analyse

- aucun accès disque ;
- aucun effet de bord ;
- test rapide.

Exemple : Mock avec vérification

Mock manuel

```
class TestLogger implements Logger {  
    int count = 0;  
    public void log(String msg) {  
        count++;  
    }  
}
```

Test

```
TestLogger logger = new TestLogger();  
LogicielDeCalcul l =  
    new LogicielDeCalcul(logger);  
l.calcul(10);  
assert logger.count > 0;
```

Intérêt

- vérification du comportement ;
- test du contrat ;
- isolation complète.

Frameworks de mocks

Constat

Écrire des mocks à la main devient rapidement fastidieux.

Solution

Utilisation de frameworks dédiés comme Mockito ;

```
@Autowired @MockBean
MessageService messageService;

@Autowired
MockMvc mockMvc;

@Test
public void testListeVide() throws Exception {
    when(messageService.findAll()).thenReturn(Collections.emptyList());

    mockMvc.perform(get("/")).andExpect(status().isOk())
        .andExpect(view().name("list"))
        .andExpect(model().attribute("messages", Collections.emptyList()));
}
```


À retenir

- inversion de dépendances ;
- injection de dépendances ;
- testabilité ;
- automatisation par framework.

Message clé

Un framework d'injection automatise un principe de conception, il ne le remplace pas.

Un bon design est un code qui ne sait pas comment ses dépendances sont créées.

Section 5

Exemple de framework : Guice

Guice

- Framework d'injection de dépendances pour Java
- Léger, non intrusif
- Basé sur annotations et configuration Java

Principe fondamental

Guice implémente l'**Inversion de Contrôle**

Injection par constructeur

Classe avec dépendance

```
public class OrderService {  
    private final Database db;  
  
    @Inject  
    public OrderService(Database db) {  
        this.db = db;  
    }  
}
```

Pourquoi le constructeur ?

- Dépendances explicites
- Classe valide dès sa création
- Favorise l'immuabilité

Injection par méthode

Setter injection

```
public class OrderService {  
    private Database db;  
  
    @Inject  
    public void setDatabase(Database db) {  
        this.db = db;  
    }  
}
```

Usage

- Dépendances optionnelles
- Configuration tardive

Module

```
public class AppModule extends AbstractModule {  
    @Override  
    protected void configure() {  
        bind(Database.class)  
            .to(MySQLDatabase.class);  
    }  
}
```

Rôle

- Définir les implémentations concrètes
- Centraliser l'assemblage

@Provides : création contrôlée des dépendances

Module avec @Provides

```
public class AppModule extends AbstractModule {  
  
    @Provides  
    @Singleton  
    Database provideDatabase(Config config) {  
        if (config.isTest()) {  
            return new InMemoryDatabase();  
        }  
        return new MySQLDatabase();  
    }  
}
```

Rôle de @Provides

- Déclare un **binding**
- Autorise de la **logique**
- Paramètres injectés par Guice

À retenir

@Provides exprime la **logique d'assemblage** sans casser l'IoC.

Création de l'injecteur

Bootstrap

```
Injector injector =  
    Guice.createInjector(new AppModule());  
  
OrderService service =  
    injector.getInstance(OrderService.class);
```

Hollywood Principle

L'application ne crée plus ses objets.

Vers une instance

```
bind(Database.class)  
    .toInstance(new InMemoryDatabase());
```

Vers un Provider

```
bind(Database.class)  
    .toProvider(DatabaseProvider.class);
```

Singleton

```
@Singleton  
public class Database { }
```

Scopes disponibles

- Par défaut : nouvelle instance
- Singleton
- Scopes personnalisés

Named Bindings

Configuration

```
bind(Database.class)
    .annotatedWith(Names.named("prod"))
    .to(MySQLDatabase.class);
```

Injection

```
@Inject
public Service(@Named("prod") Database db) {}
```

Optional Bindings

Injection optionnelle

```
@Inject(optional = true)  
private Logger logger;
```

Usage

- Extensions
- Plugins
- Configuration conditionnelle

Limites

- Pas d'écosystème complet
- Configuration manuelle
- Pas orienté entreprise

Mais

Guice est idéal pour comprendre les bases de l'IoC.